



Report for the Course Cyber-Physical Systems and IoT Security

Reference paper: SIMPLE – A Remote Attestation Approach for Resource-constrained IoT devices

Andrea Mutti
Stefano Masiero

2026-09-24

1 Objectives

Remote attestation is a security mechanism that allows a trusted verifier to assess whether the software state of a remote device is still consistent with an expected benign configuration. This problem is particularly relevant in cyber-physical systems and IoT deployments, where low-end devices often execute security-critical functions while operating with limited computing resources and restricted hardware support.

In this context, the reference paper is especially relevant because it proposes an attestation design intended to bridge the gap between purely software-based and hybrid remote attestation approaches for constrained embedded platforms [Ammar et al., 2020].

1.1 Project goal and deviations from the paper

This project does not aim to reproduce exactly the original SIMPLE implementation. Its goal is instead to implement an adapted prototype that preserves the core verifier-prover attestation workflow while mapping it onto a different hardware architecture based on a PC verifier, an ESP32 gateway, and an STM32 prover. While the prototype retains the main protocol-level ideas of SIMPLE, it differs from the original design in its hardware platform, communication architecture, and trusted computing assumptions. In particular, the reference paper relies on $S\mu V$ -based software isolation for resource-constrained devices, whereas the present implementation is built on an STM32-based platform with an ESP32 communication gateway.

2 Background: remote attestation

This section summarises the remote attestation concepts and the specific mechanisms from the SIMPLE paper that the implementation builds on: the verifier-prover model, the spectrum of attestation approaches, and the software-based isolation layer that SIMPLE relies upon.

2.1 The verifier-prover model

Remote attestation is an interactive protocol between two parties: a trusted *verifier* and an untrusted remote *prover*. The verifier issues a challenge containing a fresh nonce (or counter) and the expected representation of the prover's software state. The prover computes a cryptographic digest of its own memory, compares it with the expected value, and returns an authenticated report indicating whether the two match. If they do, the verifier considers the prover's software integrity intact; otherwise, a compromise is reported.

Two properties make the protocol meaningful. First, **freshness**: a monotonic counter and a nonce prevent an adversary from replaying an old, valid report. Second, **authenticity**: keyed-hash message authentication codes (HMACs) on both request and response guarantee that only the legitimate prover, holding the correct secret keys, can produce a valid report.

2.2 Attestation approaches

Prior work on remote attestation falls along a spectrum of hardware requirements [Ammar et al., 2020].

- **Software-based** techniques assume no hardware security features and rely on tight timing constraints or memory filling to detect tampering. They offer no formal security guarantees in a remote, multi-hop setting and are vulnerable to attacks such as return-oriented programming (ROP) and time-of-check-to-time-of-use (TOCTOU) races.
- **Hardware-based** techniques depend on a dedicated trust anchor, such as a Trusted Platform Module (TPM), to protect attestation code and keys. They provide strong guarantees but are too costly for Class-1 IoT devices with as little as 10 kB of RAM and 100 kB of Flash.
- **Hybrid** techniques impose a minimal set of hardware features, typically read-only memory (ROM) and a memory protection unit (MPU), to isolate attestation code while keeping cost low. They are the current standard for

embedded attestation, but they require that the device either already contains or can be extended with these hardware primitives.

2.3 SIMPLE and the Security MicroVisor

SIMPLE bridges the gap between software-based and hybrid attestation. It targets devices that lack both a TPM and a hardware MPU, yet need concrete security guarantees stronger than what pure software techniques offer. Its key idea is to replace the hardware MPU with a formally verified software isolation layer called the *Security MicroVisor* ($S\mu V$) [Ammar et al., 2020].

$S\mu V$ reserves a protected memory region for a Trusted Computing Module (TCM) that stores the attestation code, the secret keys, and the monotonic counter. All application code is verified at load time to ensure it cannot read, write, or execute anything inside the TCM, except through hard-coded entry points. Global interrupts are disabled during attestation, guaranteeing atomic execution. The combination of memory safety, crash freedom, and atomicity provides the same isolation properties that a hardware MPU would enforce, but entirely in software.

The protocol itself uses two symmetric keys with distinct roles. K_{auth} is shared between verifier and prover and authenticates both requests and responses via HMAC-SHA256. K_{attest} is held only by the prover and is used to derive the *valid-state* (VS) digest from the measured memory regions. This separation avoids overloading a single secret with multiple cryptographic functions and makes the security argument modular: a valid protocol MAC proves the message originated from a key holder, while a matching VS proves the prover’s memory content is consistent with the verifier’s reference.

3 System Setup

The project was implemented on a three-node architecture composed of a PC, an ESP32, and an STM32 Nucleo-H503RB.

3.1 System architecture

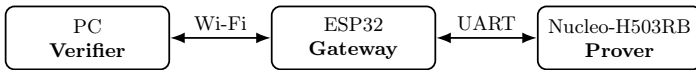


FIGURE 1: Hardware architecture of the project: the PC verifier communicates with the STM32 prover through an ESP32 Wi-Fi/UART gateway.

Figure 1 shows that the PC operates as verifier and runs the software responsible for issuing attestation requests and checking the authenticity and correctness of the received responses. The Nucleo-H503RB acts as prover and executes the firmware that performs the attestation procedure. The ESP32 is placed between the two devices and serves as a communication gateway, forwarding messages from the wireless interface used by the verifier to the serial interface connected to the prover.

This organization was adopted to obtain a modular and easily observable testbed. In particular, the PC–ESP32 link uses Wi-Fi, while the ESP32–STM32 link uses UART. As a result, the gateway does not implement the attestation logic itself, but only relays protocol messages between the

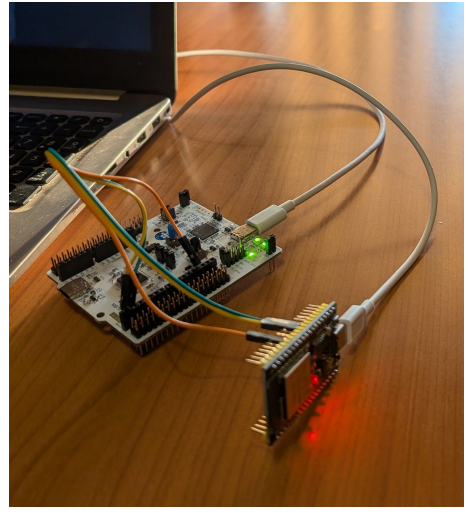


FIGURE 2: Physical testbed used for the prototype.

verifier and the prover. This choice simplifies integration and debugging, while preserving the verifier–prover interaction model of the original protocol.

3.2 Verifier

The verifier was implemented as a Python command-line application running on the host PC. Its role is to initiate the attestation procedure, generate authenticated requests, maintain the expected reference values for each attestation profile, and validate the responses returned by the prover. In the implemented setup, the verifier connects to the ESP32 gateway over TCP, which then relays the messages to the STM32 prover through UART. The verifier therefore acts as the central control point of the experimental workflow.

The verifier stores persistent local state in a JSON file. In particular, it records the last accepted counter value and one expected valid-state value for each supported attestation profile. The local state is indexed by communication endpoint, allowing the verifier to maintain separate reference values for different prover instances if needed. This mechanism is important because attestation requests must always use a counter greater than the previously accepted one, and because the received digest must be compared against a stored reference baseline rather than against a value generated at runtime.

A relevant feature that we added to simplify the verifier implementation is the bootstrap mode used during initialization. In this mode, the verifier first transmits an attestation request with a null expected valid-state value, receives the prover-computed local valid state, and then repeats the attestation using that returned value as the expected reference. If the second attestation succeeds, the verifier stores the learned value locally and uses it as the baseline for future runs.

This mechanism is convenient for a laboratory prototype because it simplifies enrollment and testing. However, it is important to emphasize that such a mechanism should not be present in a real-world deployment, since the reference baseline should be provisioned through a trusted process rather than learned from the prover itself.

LISTING 1: Example of verifier-side persistent state stored in JSON format. For each target endpoint, the verifier keeps the last accepted counter and one expected valid-state value for each attestation profile.

```
{
  "192.168.4.1:8080": {
    "last_counter": 42,
    "profiles": {
      "1": {
        "name": "flash-only",
        "expected_vs_hex": "6f8c3d8d2d8b1d2d4a79b3d9f8d8f43c1e7f9c2a14b2d5"
      },
      "2": {
        "name": "flash+ram",
        "expected_vs_hex": "1c9a77d3b02f4e9d8aa31c7780d22b5fd4a2d61b93cc42"
      },
      "3": {
        "name": "flash+ram+cfg",
        "expected_vs_hex": "95c41e2a76b54fd91cc8d6a1a7bb3e2fd23ac6b6ee42a17"
      }
    }
  }
}
```

3.3 Gateway

The gateway was implemented on an ESP32 board and configured as an intermediate communication node between the verifier and the prover. Its function is purely infrastructural: it exposes a Wi-Fi access point to the PC-side verifier and forwards the exchanged messages to the STM32 prover through a UART link. In the implemented setup, the ESP32 access point is used as the network entry point of the attestation system, while the serial interface provides a simple and reliable connection to the prover firmware.

At the software level, the ESP32 executes a lightweight TCP–UART bridge implemented using the ESP-IDF software framework provided by [Espressif Systems, 2026]. The verifier opens a TCP connection to the gateway, and the ESP32 transparently relays the received bytes to the STM32 over UART. The same mechanism is used in the reverse direction, so that responses generated by the prover are sent back to the verifier through the same path. In the current implementation, the gateway listens on port 8080 and uses a UART configuration at 115200 baud. As a result, the ESP32 does not parse, interpret, or modify the semantics of attestation messages, but only transfers them between the two communication domains.

From a security perspective, the ESP32 is not intended to act as a trusted component of the attestation logic. Authentication of requests and responses remains end-to-end between verifier and prover, and the gateway does not possess the cryptographic role required to produce valid attestation results. Therefore, the ESP32 should be regarded as a relay node rather than as part of the verifier–prover trust anchor.

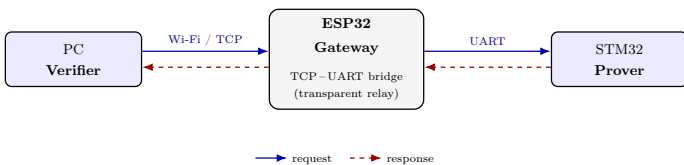


FIGURE 3: ESP32 gateway used as a transparent TCP–UART relay between verifier and prover. The gateway forwards messages but does not implement attestation logic or hold cryptographic trust.

3.4 Protocol and message structure

From the implementation viewpoint, the verifier generates a fresh nonce for each attestation request, maintains a monotonic counter to enforce freshness, and builds request frames according to a custom binary protocol. Each request includes a counter, a profile identifier, a nonce, an expected valid-state value, and an HMAC-based authentication tag. Upon receiving a response, the verifier checks frame integrity, validates the authentication tag, verifies that the returned counter and profile identifier are consistent with the transmitted request, and finally interprets the attestation outcome.

At the transport level, the protocol uses a custom binary frame format over the TCP–UART path between verifier and prover. Each frame begins with two start-of-frame bytes (0xA5 and 0x5A), followed by a protocol version, a message type, a payload length, the payload itself, and a CRC-16 value. The start-of-frame bytes help the receiver detect frame boundaries and resynchronize the parser if needed, while the length field indicates how many payload bytes must be collected before processing.

This framing format mainly supports message delimitation and transmission error detection. In particular, the CRC-16 field is used to detect accidental corruption on the communication path, for example on the UART link or during gateway forwarding. The specific variant used by both endpoints is CRC-16/CCITT-FALSE, with initial value 0xFFFF, polynomial 0x1021, no input or output reflection, and no final XOR; this avoids any ambiguity between the verifier and prover implementations, since several mutually incompatible variants are commonly referred to as “CRC-16”. However, CRC-16 is not a security mechanism: end-to-end authenticity and cryptographic integrity are provided by the HMAC tag included in the attestation payloads.

3.4.1 Request and response payloads

Figure 4 summarizes the structure of the attestation request and response payloads used in the prototype.

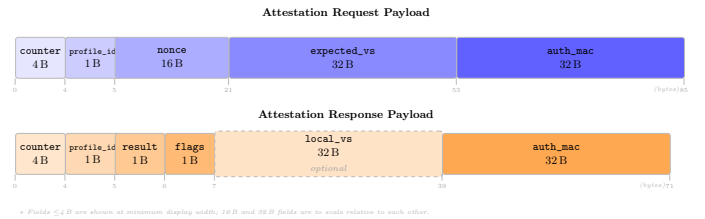


FIGURE 4: Attestation request and response payloads. Requests carry counter, profile, nonce, expected valid state, and authentication MAC; responses return the result, flags, optional local_vs, and response MAC.

3.5 Prover

The prover was implemented as firmware running on an STM32 Nucleo-H503RB board [STMicroelectronics, 2026]. Its role is to receive attestation requests from the verifier through the ESP32 gateway, validate their authenticity and freshness, compute a profile-dependent representation of the local software state, compare the resulting value against the expected reference provided by the verifier, and return an authenticated response.

A relevant implementation detail is that the attestation logic is not executed directly in the normal application context. After a request has been parsed successfully, the unprivileged execution context invokes the attestation routine through an SVC-based dispatch path. The request is first copied into a privileged mailbox located in protected RAM, then processed there, and finally the result is copied back to the unprivileged stage buffer before transmission. After every transaction, the privileged mailbox is explicitly zeroized.

3.6 Prover memory organization

The implemented verifier supports three attestation profiles:

- flash-only;
- flash+ram;
- flash+ram+cfg.

These labels correspond to progressively broader measurements on the prover side. More precisely, inspection of the STM32 firmware shows that Profile 1 measures the attested flash region together with a privileged flash region reserved for attestation-related code and constants. Profile 2 extends this measurement by including a dedicated RAM section, rather than the whole SRAM space; the protected RAM regions `.attest_state`, `.attest_secure`, and `.attest_mailbox` are deliberately excluded from every profile because their contents change at runtime (lifecycle flags, the monotonic counter, and the cleared request/response mailbox respectively), so measuring them would yield a different digest on every attestation. The dedicated section that is measured, `.app_attested_ram`, contains a small application-defined manifest with a magic value, a layout version, and the supported protocol and profile masks, as declared in `app_attested_state.c`; this content is stable across runs and is initialised from flash at startup, so the prover’s RAM image of the manifest tracks the flashed binary. Profile 3 further includes a protected configuration-shadow region refreshed by the prover before digest computation. Consequently, the verifier profiles should be interpreted as references to specific memory regions selected by the prover firmware, not as generic measurements of the full flash or RAM contents.

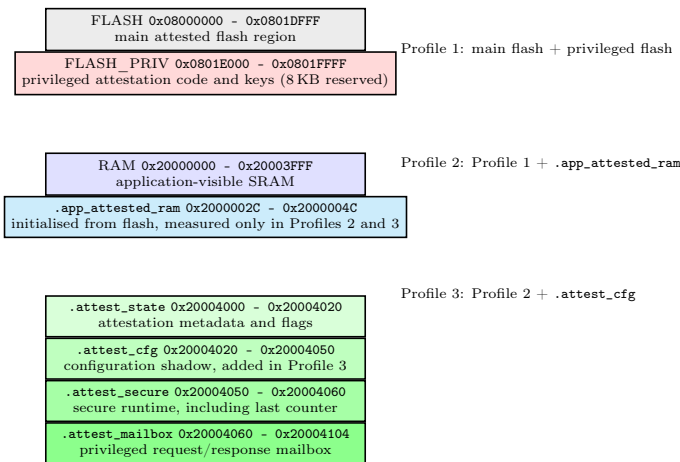


FIGURE 5: STM32 prover memory layout and profile coverage. Profile 1 measures main flash and privileged flash; Profile 2 adds `.app_attested_ram`; Profile 3 also includes the runtime configuration shadow `.attest_cfg`.

3.7 Protection mechanisms

The prover firmware configures hardware-supported protection mechanisms available on the STM32H503xx platform. In particular, the MPU is used to distinguish between application-visible memory and privileged overlays, GTZC is configured to protect the HASH peripheral and the privileged SRAM region, and a reserved flash window is marked as privileged for attestation-related code and constants.

On top of these memory and peripheral-level restrictions, the firmware also relies on the Cortex-M33 privilege model to bound where the attestation logic can execute. After all peripherals and protection structures have been initialised, the function `app_enter_unprivileged_mode()` sets the `nPRIV` bit of the `CONTROL` register, so the main loop and all normal request handling run in unprivileged Thread mode. Because the MPU is configured with privileged-only attributes on the `FLASH_PRIV` window and on the privileged SRAM region, the attestation code, the symmetric keys, the secure counter, and the privileged mailbox cannot be read, written, or executed directly from this unprivileged context. The only way to invoke attestation is through the `svc 0` instruction issued by the application stub, which raises an SVC exception and transfers control to the `SVC_Handler`; this handler runs in Handler mode, which is always privileged, and dispatches to `app_attestation_svc_dispatch()` placed inside the privileged flash region. The MPU and GTZC therefore enforce the spatial separation of code and data, while the privilege transition through `svc 0` is the actual mechanism that gates entry into the trusted attestation routine.

This is a major architectural difference with respect to the original SIMPLE design: the present prototype relies on STM32 hardware protection features and therefore does not reproduce the same trusted computing assumptions as the software-isolated design discussed in the paper.

4 Experiments

This section describes the methodology used to implement and evaluate the adapted attestation workflow.

4.1 Implemented attestation flow

The attestation workflow implemented in the project is a single-prover challenge-response protocol. The sequence of operations is the following:

1. The verifier generates a fresh nonce and selects a monotonic counter value greater than the last accepted one.
2. The verifier selects one of the three supported profiles and loads the corresponding expected software-state value.
3. The verifier constructs an authenticated request containing the counter, profile identifier, nonce, expected state, and an HMAC-based authentication tag.
4. The request is sent over Wi-Fi to the ESP32 gateway, which forwards it over UART to the STM32 prover.
5. The STM32 parses the request, verifies the authentication tag, checks whether the received counter is fresh, computes its local software-state value for the selected profile, compares it against the expected value, and prepares an authenticated response.

6. The response is returned through the same UART and Wi-Fi path and is finally verified by the PC verifier.

This flow preserves the central protocol intuition of SIMPLE, namely that a trusted verifier checks the integrity of a remote device by means of an authenticated and freshness-protected exchange. However, the implementation also includes adaptations introduced for course-project feasibility. In particular, the verifier supports a bootstrap mode in which the local software-state value returned by the prover can be learned and stored as the expected reference value for subsequent runs.

4.2 Prover internal processing

After a request has been received, the STM32 prover processes it through a structured internal pipeline. First, the UART parser reconstructs the frame and verifies its CRC-16 integrity field. If the frame is valid, the request is handed from the unprivileged execution context to a privileged SVC path. Within this protected path, the request MAC is verified, the freshness counter is checked, and the attestation profile is mapped to the corresponding set of measured regions.

The measured state is not obtained by hashing a single contiguous memory image. Instead, the firmware first maps the selected profile to a set of disjoint regions. Each region is hashed individually with SHA-256 using the STM32 HASH peripheral. The prover then concatenates the profile identifier and the region digests and applies HMAC-SHA256 under the attestation key `K_ATTEST` to derive the final 32-byte valid-state value. This value is compared in constant time with the `expected_vs` field provided by the verifier. If the two values match, the prover returns a success code; otherwise, it returns an attestation-failure code. Request and response authentication are instead computed with a distinct key, `K_AUTH`, again through HMAC-SHA256.

The prototype therefore uses two logically distinct symmetric keys with different security roles. `K_AUTH` is shared by the verifier and the prover and is used exclusively for request and response authentication, thus providing end-to-end integrity and source authenticity for protocol messages exchanged between the two endpoints. `K_ATTEST`, instead, is embedded only in the prover firmware and is used by the prover to compute the final valid-state value derived from the measured memory regions. This separation avoids overloading a single cryptographic secret with multiple functions and makes the design easier to reason about from a security perspective. In particular, a valid protocol MAC proves that the message was generated by an entity holding `K_AUTH`, while a matching valid-state value shows that the prover-computed state is consistent with the verifier’s stored reference baseline.

In the current prototype, `K_AUTH` is statically embedded in both the verifier and the prover, while `K_ATTEST` is statically embedded only in the prover firmware. The verifier does not independently recompute the valid-state value under `K_ATTEST`; instead, it stores an `expected_vs` value learned during bootstrap and later uses it as an opaque reference for comparison. This is a deliberate simplification with respect to the original SIMPLE trust model, where the verifier is assumed to know the expected valid state through a trusted provisioning process. The current design is acceptable for a laboratory proof of concept, but it would not be suitable for production without secure key provisioning, device-bound

secrets, and a trusted mechanism for installing the reference baseline.

Freshness is enforced by storing the last accepted counter in a protected runtime structure located in privileged RAM. After verifying the request MAC, the prover checks whether the received counter is strictly greater than the stored one. Requests carrying stale counters are rejected with a dedicated result code. It is worth noting a non-obvious design choice in the prover firmware: once the request has been authenticated and the counter has been accepted as fresh, the stored counter is updated to the new value *before* the local valid state is compared against `expected_vs`. As a consequence, even an attestation that returns `ATTESTATION_FAILURE` due to a state mismatch still “consumes” that counter value. This behaviour strengthens replay protection, where an adversary cannot reuse an old request frame, regardless of whether the original exchange succeeded or failed, but it also means that the secure counter advances monotonically across all authenticated requests, not only across successful ones. The firmware also supports a debug mode, enabled in the current codebase, in which the locally computed valid-state value can be included in the response. This feature is useful during development and bootstrap, but it should be interpreted as a prototype-oriented design choice rather than as a production-grade security feature.

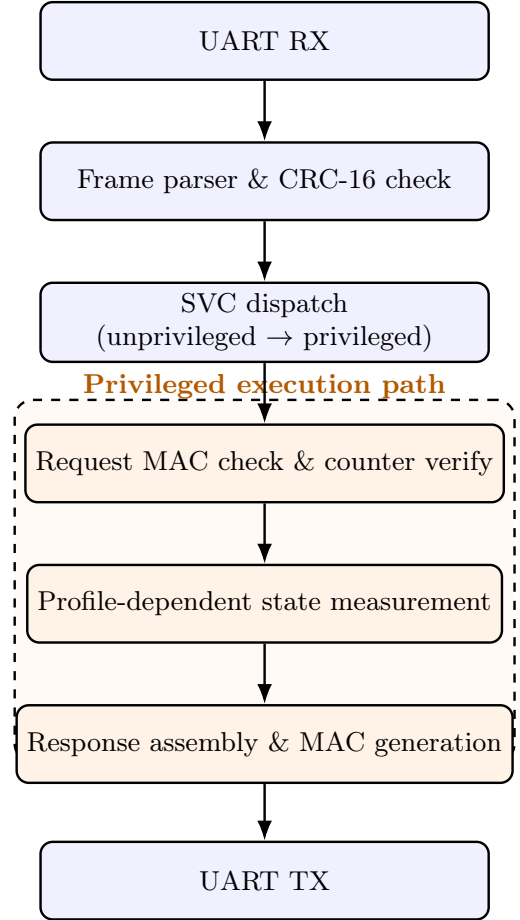
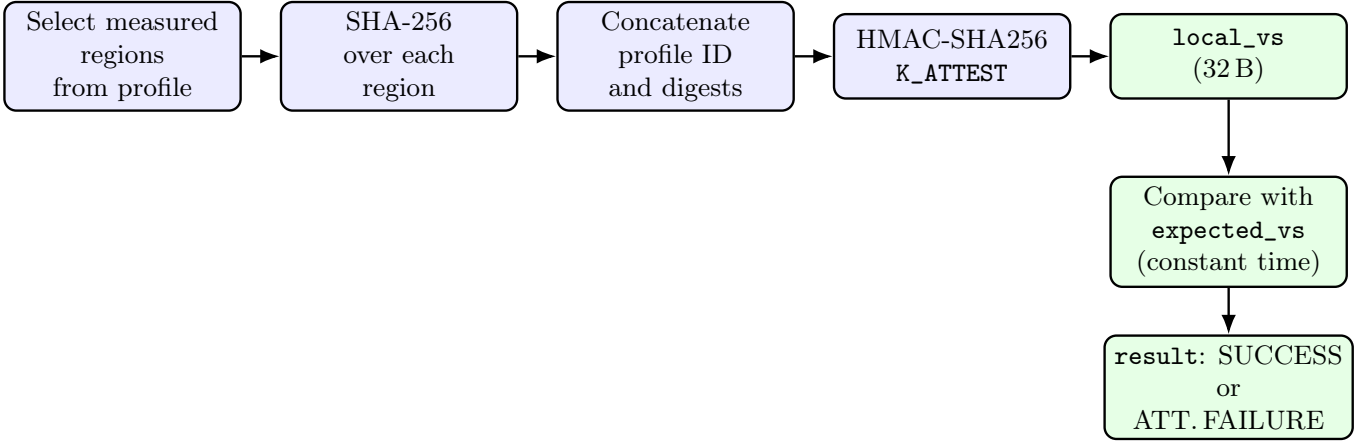


FIGURE 6: Internal processing flow of the STM32 prover. Parsed requests enter the privileged SVC path, where MAC verification, freshness checking, state measurement, and response generation are performed.

State derivation branch



Protocol authentication branch

Mandatory inputs (always signed):

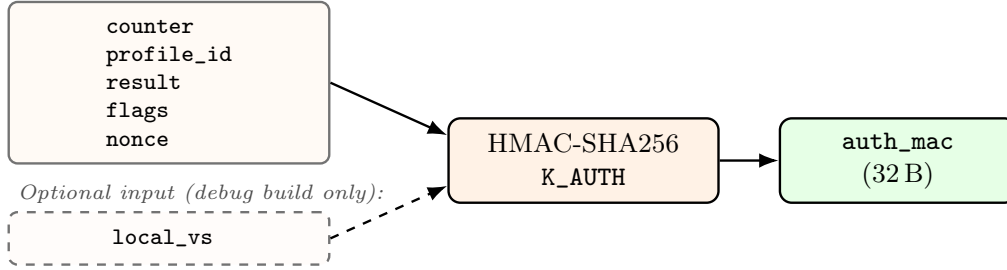


FIGURE 7: Separation between state derivation and protocol authentication. K_ATTEST is used to compute local_vs from measured regions, while K_AUTH authenticates the protocol response.

4.3 Functional tests

The prototype was validated through a set of functional tests aimed at checking baseline enrollment, successful attestation in the benign case, detection of state modifications, and freshness enforcement.

4.3.1 Bootstrap and baseline enrollment

For each profile, the verifier was first executed in bootstrap mode. In this configuration, the verifier sends an attestation request with a null expected valid-state value, receives the prover-computed local_vs, and immediately retries using that value as the expected reference. A successful second exchange confirms that the enrolled value can be used as a baseline for subsequent attestations.

LISTING 2: Example bootstrap command used to enroll the reference value for Profile 1.

```
python3 -m verifier --profile 1 --bootstrap
```

LISTING 3: Representative bootstrap output. The first request learns the current local_vs; the second request validates it and stores it locally.

```
[1] bootstrap request for profile 1 (flash-only)
[1] nonce=...
[1] counter=1 profile=1 result=0xe4(ATTTESTATION_FAILURE) flags=0x01
[1] local_vs=<hex>

[2] retry with learned local_vs
[2] nonce=...
[2] counter=2 profile=1 result=0x00(SUCCESS) flags=0x01
```

```
[2] local_vs=<same hex>
```

```
Stored expected VS for profile 1: <hex>
```

4.3.2 Successful attestation in the benign case

After bootstrap, a normal attestation was executed without changing the prover firmware or runtime state. In this case, the verifier returned a success result, confirming that the measured prover state matched the stored reference for the selected profile.

LISTING 4: Example command for a standard attestation run after baseline enrollment.

```
python3 -m verifier --profile 1
```

LISTING 5: Representative successful attestation output.

```
Requesting attestation for profile 1 (flash-only)
counter=3
nonce=...
expected_vs=<hex>
[resp] counter=3 profile=1 result=0x00(SUCCESS) flags=0x01
[resp] local_vs=<hex>
Attestation OK.
```

4.3.3 Detection of adversarial modifications

To emulate post-deployment tampering, we intentionally modified memory content belonging to the attested regions and repeated attestation using the previously enrolled reference value. Three classes of modifications were considered:

- changes in the main attested flash region;
- changes in the dedicated attested RAM region;
- changes in the protected configuration-shadow region.

As expected from the profile design, modifications in the main flash region are detected already by Profile 1, modifications in `.app_attested_ram` are detected by Profiles 2 and 3, and modifications in `.attest_cfg` are detected by Profile 3 only. It is worth recalling that Profile 1 does not measure a single contiguous flash blob: it hashes two disjoint regions, namely the *main attested flash* window, which contains the application code, the protocol parser, the HAL drivers, and other read-only data of the regular firmware, and the *privileged flash* window, which is reserved for the attestation routines, the request/response handlers, and the static `K_AUTH` and `K_ATTEST` keys. The two digests are then combined under `K_ATTEST` together with the profile identifier to produce `local_vs`, so a tamper in either region individually changes the final valid-state value. In all such cases, the verifier reported an attestation-failure result because the prover-derived `local_vs` no longer matched the stored baseline.

A representative workflow for a flash-region modification is shown below. After enrolling the baseline, one constant or code fragment within the measured flash area was intentionally changed, the prover firmware was rebuilt and reflashed, and attestation was executed again with the original verifier-side reference.

LISTING 6: Example command used after reflashing a modified prover image.

```
python3 -m verifier --profile 1
```

LISTING 7: Representative output after a modification in an attested region. The response now reports a mismatch with respect to the previously stored baseline.

```
Requesting attestation for profile 1 (flash-only)
counter=4
nonce=...
expected_vs=<old baseline>
[resp] counter=4 profile=1 result=0xe4(ATTESTATION_FAILURE) flags=0x01
[resp] local_vs=<new hex>
Attestation FAILED: expected VS does not match the prover state.
```

These tests show that the implemented attestation logic does not merely authenticate the exchange, but also detects semantic changes in the measured state. In particular, the protocol reports failure even when communication remains well-formed and correctly authenticated, provided that the measured memory content differs from the enrolled benign baseline.

4.3.4 Freshness and replay protection

Replay resistance was tested by intentionally reusing a previously accepted counter value. In this case, the prover rejected the request with a stale-counter result, showing that the protected counter stored in privileged memory is effectively used to prevent acceptance of replayed attestation requests.

LISTING 8: Example command forcing reuse of an already accepted counter value.

```
python3 -m verifier --profile 1 --counter 3
```

LISTING 9: Representative output for a replayed request using a stale counter.

```
Requesting attestation for profile 1 (flash-only)
counter=3
nonce=...
expected_vs=<hex>
[resp] counter=3 profile=1 result=0xe6(STALE_COUNTER) flags=0x00
Request rejected because the counter is stale.
```

Overall, these tests do not prove resistance against a strong physical adversary, but they do show that the implemented prototype correctly enforces authenticated challenge-response attestation, profile-dependent state measurement, mismatch detection, and counter-based freshness.

4.4 Code availability

The complete firmware and verifier source are provided with the submission. The exact version used for these experiments is published at:

https://github.com/stefano-masiero/Remote_Attestation_SIMPLE

5 Results and Discussion

The implementation demonstrates that the core verifier-prover workflow of remote attestation can be reproduced on a heterogeneous setup composed of a PC verifier, an ESP32 relay, and an STM32-based prover. From an architectural perspective, the prototype preserves the essential attestation structure of a challenge-response exchange protected by message authentication and freshness checks, while adapting it to a platform with different hardware assumptions from those of the original SIMPLE design.

A first relevant result is that the prototype supports profile-dependent attestation over multiple classes of memory regions. Rather than measuring a single monolithic image, the prover maps each profile to a set of selected regions and derives a final valid-state value from their individual digests. This makes it possible to distinguish between different integrity scopes, such as flash-only attestation and attestation that also includes selected RAM or configuration state. The corresponding tests show that changes affecting different regions are detected exactly by the profiles that include them, which confirms the internal consistency of the measurement design.

A second relevant result is the separation between state binding and protocol authentication. The use of `K_ATTEST` for deriving the valid-state value and `K_AUTH` for authenticating requests and responses clarifies the roles of the two cryptographic operations and avoids conflating protocol integrity with memory-state integrity. This separation is useful from a design perspective because it makes the attestation logic easier to reason about and easier to extend.

5.1 Runtime measurements

To complement the functional validation, we measured the execution time of the cryptographic operations performed inside the privileged attestation path of the STM32 prover. Measurements were collected using the Cortex-M cycle counter and averaged over 100 successful attestation runs for each profile. Bootstrap executions were excluded from the measurement set. Since the ESP32 gateway only acts as a transparent TCP-UART relay, the values reported in Table 1 refer only

to prover-side computation and do not include PC-side communication latency, Wi-Fi transmission, UART forwarding, or gateway buffering.

Each runtime measurement in Table 1 corresponds to one stage of the prover-side attestation routine. Request HMAC verification checks the authenticity of the incoming challenge under K_{AUTH} ; state derivation computes the profile-dependent local valid-state value from the selected memory regions; constant-time comparison checks this value against the verifier-provided reference; response HMAC generation authenticates the reply; and the complete privileged attestation path reports the overall time spent inside the protected SVC-dispatched routine.

TABLE 1: Cryptographic runtime measurements on the STM32 Nucleo-H503RB prover. Values are internal cycle-counter measurements averaged over 100 successful attestations; all values are in milliseconds.

Operation	Profile 1	Profile 2	Profile 3
Request HMAC verify	0.0184	0.0184	0.0184
State derivation	0.8888	0.8937	0.9029
Constant-time compare	0.0033	0.0033	0.0033
Response HMAC gen.	0.0097	0.0097	0.0097
Full privileged path	0.9272	0.9321	0.9412

The results show that the computational cost of attestation is dominated by profile-dependent state derivation. This is expected because this step includes the SHA-256 measurements of the selected memory regions and the final HMAC-SHA256 computation under K_{ATTEST} . In contrast, request verification, response authentication, and constant-time comparison remain practically unchanged across profiles, since their input sizes do not depend on the selected memory coverage.

The total prover-side runtime increases from 0.9272 ms for Profile 1 to 0.9321 ms for Profile 2 and 0.9412 ms for Profile 3. This monotonic increase is consistent with the profile design: Profile 2 extends Profile 1 with the attested RAM manifest, while Profile 3 further includes the protected configuration-shadow region. The increase is small because the additional RAM and configuration regions are much smaller than the flash regions already covered by Profile 1. Therefore, the measurements confirm that the broader profiles introduce only limited overhead while providing a wider integrity scope.

The average end-to-end round-trip time observed at the verifier was approximately 249–250 ms, but it was not included in Table 1 because it is dominated by communication and host-side effects rather than by cryptographic computation on the prover.

TABLE 2: Indicative comparison with the cryptographic runtime measurements reported in the reference SIMPLE paper [Ammar et al., 2020]. All values are in milliseconds unless marked as N/A.

Operation	Uno	MicroPnP	STM32
State computation	890	1480	0.889–0.903
Challenge authentication	44.74	71.68	0.0184
Report authentication	44.75	71.87	0.0097
Vector aggregation	1.7	2.2	N/A
Full prover path	N/A	N/A	0.927–0.941

Table 2 compares the measured runtimes with the values reported in the reference SIMPLE paper. The comparison should be interpreted qualitatively rather than as a direct hardware benchmark. The original SIMPLE implementation was evaluated on 8-bit AVR platforms, namely Arduino Uno ATmega328P and MicroPnP ATmega1284P, whereas the present prototype runs on an STM32 Cortex-M33 platform and uses the STM32 HASH peripheral for SHA-256 and HMAC computations. Moreover, the measured operations are not identical: the reference paper reports, among others, the cost of computing an HMAC over 10 KB of memory and the cost of vector aggregation for SIMPLE+, while this prototype measures the internal steps of a single-prover attestation path.

Despite these differences, the comparison is useful to contextualize the experimental results. The STM32 prover completes its whole privileged attestation path in less than 1 ms, while the cryptographic operations reported for the AVR-based SIMPLE implementation are in the order of tens of milliseconds for message authentication and hundreds of milliseconds for memory-state computation. This difference is mainly explained by the different MCU class, clock frequency, cryptographic implementation, and use of hardware acceleration, rather than by protocol-level differences alone. The vector aggregation row is marked as not applicable because the current prototype implements only single-prover attestation and does not include the SIMPLE+ swarm aggregation phase.

5.2 Prototype limitations

Although the prototype reproduces the main verifier–prover workflow of remote attestation, it intentionally simplifies several aspects that would require stronger guarantees in a production-grade design.

First, the verifier supports a bootstrap mode in which the expected reference state is learned from the prover itself. This is useful for experimental setup, but it weakens the trust model and should be replaced by trusted provisioning in a real deployment.

Second, the current firmware is compiled with a debug-oriented option that allows the prover to return the locally computed valid-state value. This greatly simplifies validation and testing, but exposes internal attestation information that should not normally be disclosed in a hardened deployment.

Third, the cryptographic material is statically embedded in software rather than provisioned through a secure hardware-backed mechanism. In particular, K_{AUTH} is present in both the verifier and prover codebases because it authenticates protocol messages, whereas K_{ATTEST} is present only in the prover firmware because the verifier does not recompute valid-state values in this prototype. The verifier instead relies on an enrolled `expected_vs` baseline. The present implementation should therefore be interpreted as a proof of concept focused on protocol logic, protected execution structure, and profile-dependent measurement, rather than as a complete hardened attestation product.

Finally, the trust assumptions differ from those of the original SIMPLE proposal. The present implementation relies on hardware-supported protection mechanisms available on the STM32 platform, such as MPU and GTZC-based isolation, and therefore should not be interpreted as a direct reproduction of the software-isolated trusted-computing model

described in the reference paper.

6 Conclusions

Within the scope of the course project, the prototype can be considered successful because it shows end-to-end authenticated attestation, profile-dependent integrity checking, detection of state changes introduced after baseline enrollment, and counter-based replay protection. At the same time, the current system remains a laboratory proof of concept rather than a deployment-ready attestation architecture. Its main value lies in demonstrating the protocol flow, the internal separation between privileged and unprivileged execution, and the practical feasibility of remote attestation on constrained embedded hardware with explicit design trade-offs.

References

- [Ammar et al., 2020] Ammar, M., Crispo, B., & Tsudik, G. (2020). Simple: A remote attestation approach for resource-constrained iot devices. In 2020 ACM/IEEE 11th International Conference on Cyber-Physical Systems (ICCPS) (pp. 247–258).
- [Espressif Systems, 2026] Espressif Systems (2026). Esp-idf programming guide.
- [STMicroelectronics, 2026] STMicroelectronics (2026). Nucleo-h503rb.